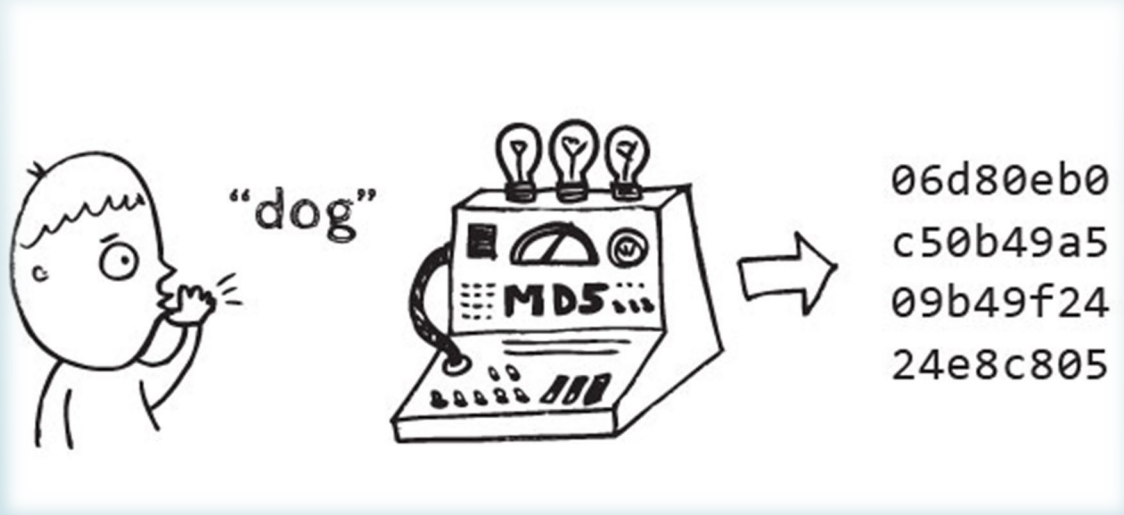


Cryptographic Hash Functions



Content of this Chapter

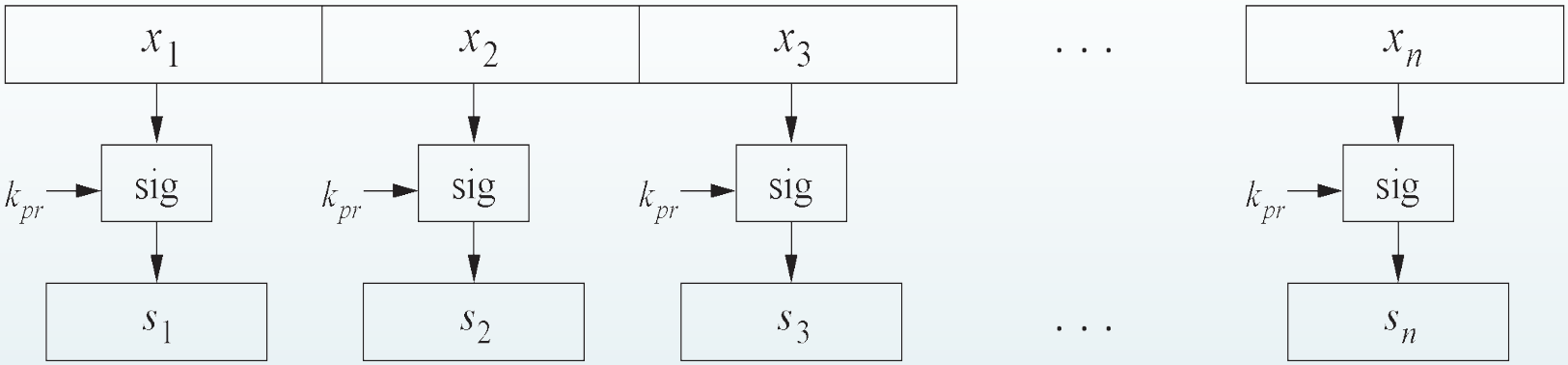
- Why we need hash functions
- How does it work
- Security properties
- Algorithms
- Example: The Secure Hash Algorithm SHA-1

Content of this Chapter

- **Why we need hash functions**
- How does it work
- Security properties
- Algorithms
- Example: The Secure Hash Algorithm SHA-1

Motivation

Naive signing of long messages generates a signature of same length.



Three Problems

- Computational overhead (256MB file need 1 Million 256bit RSA signatures) ☹
- Message overhead (256MB file, total of 512MB must be transmitted) ☹
- Security limitations (Can Replace/Remove Blocks) ☹

Solution:

Instead of signing the whole message, sign only a digest (=hash)

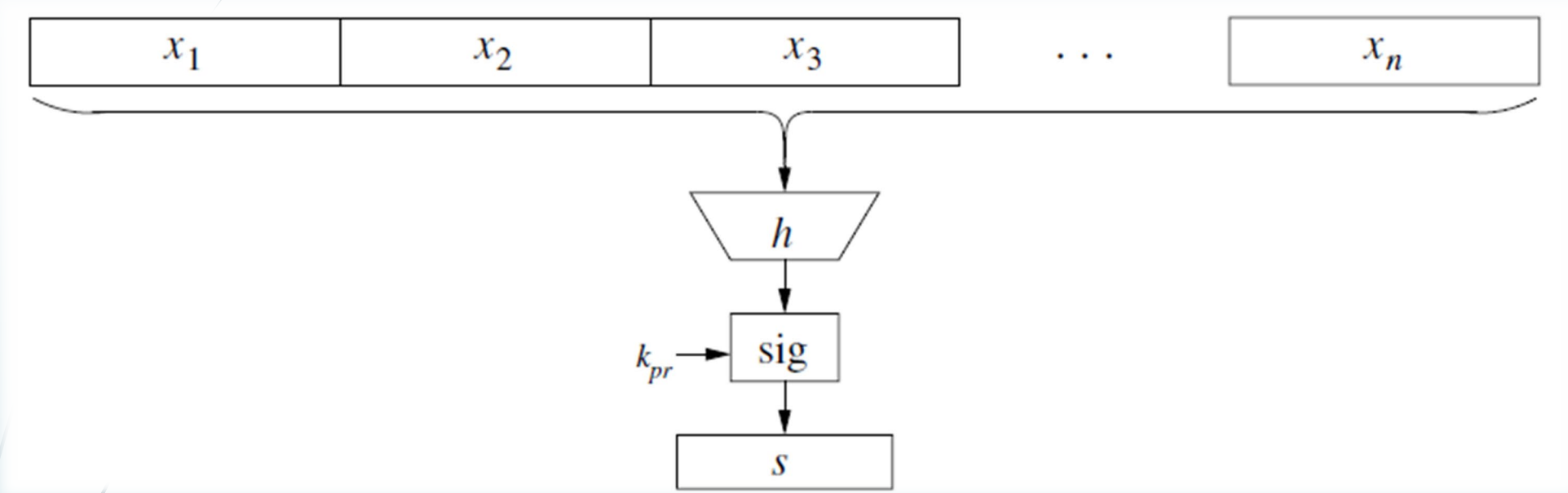
Also secure, but much faster ☺

Needed:

Hash Functions! –> One short signature for an arbitrary length message

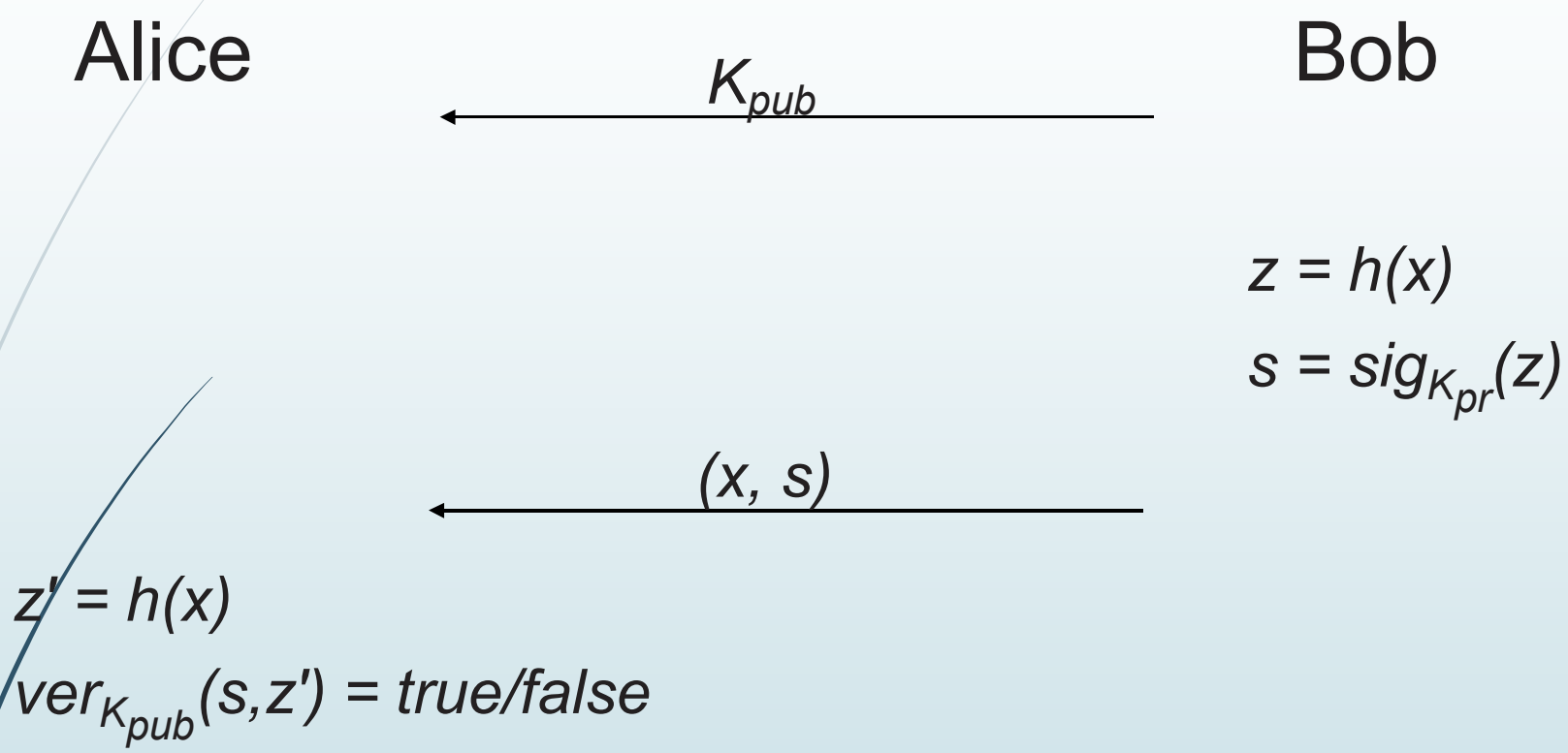


Digital Signature with a Hash Function



- If we had a hash function that somehow computes a fingerprint of the message ‘x’ we could perform the signature operation as shown above.

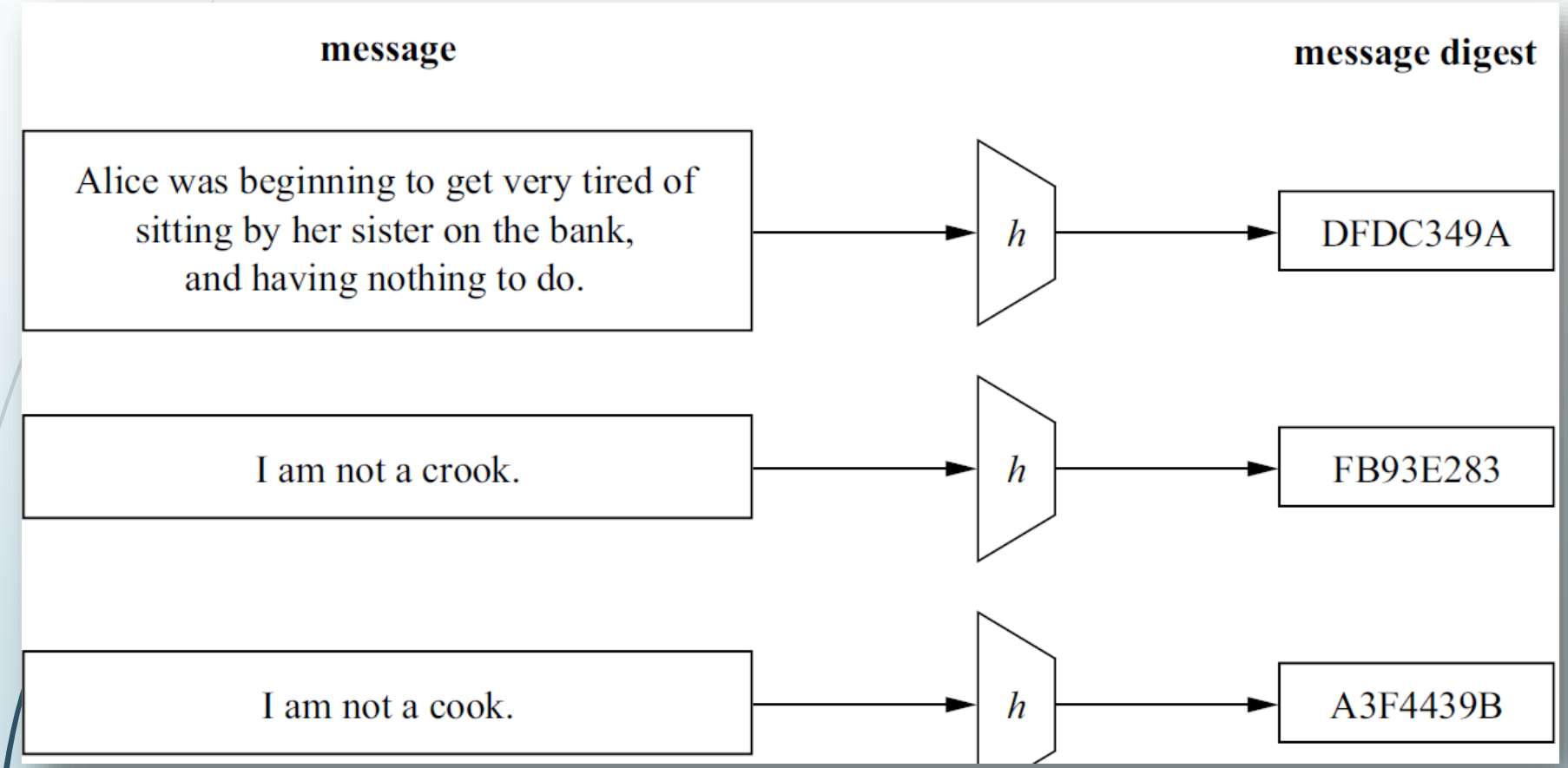
Basic Protocol for Digital Signatures with a Hash Function:



Note: Both the signature generation and the verification operate on the hash value 'z' rather than on the message itself. Hence, the hash value represents the message(Fingerprint/Digest).

Principal input–output behavior of hash functions

- Output of a hash function is of **fixed length** and independent of the input length.



- Practical hash functions have output lengths between **128–512 bits**.
- **Diffusion** property is desirable as well as **Efficiency**.

Content of this Chapter

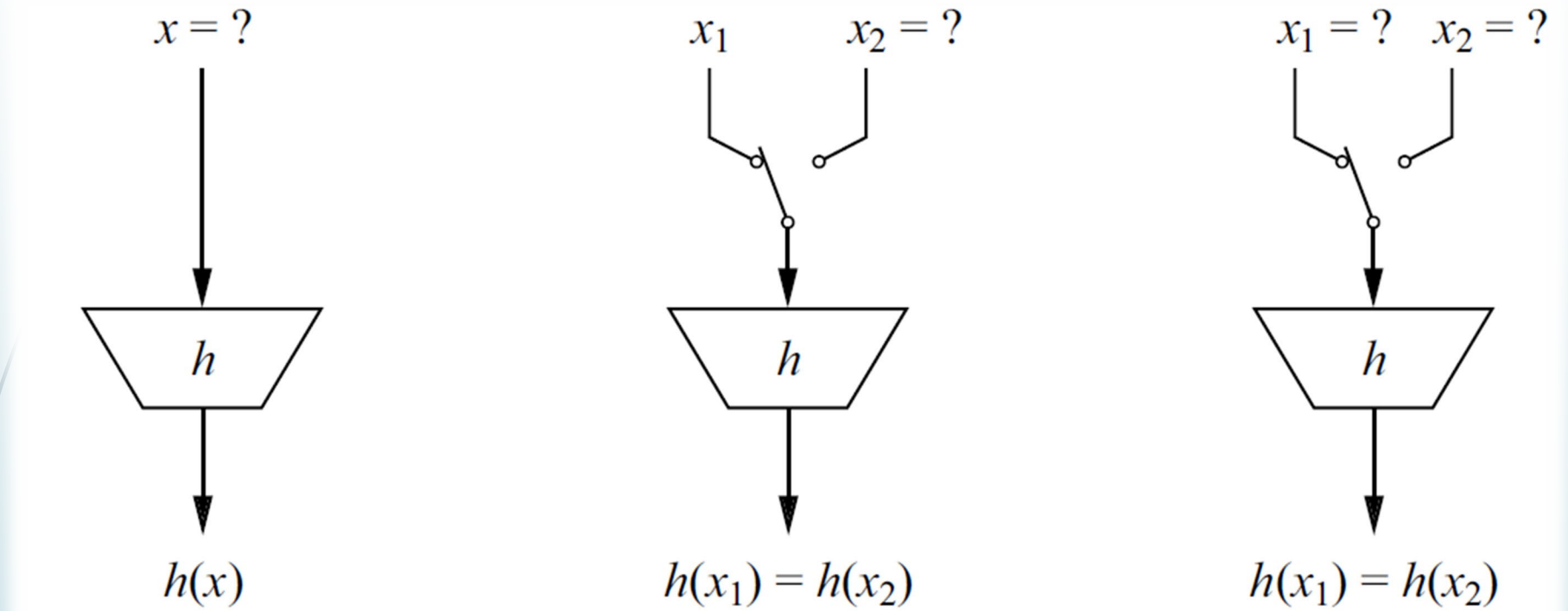
- Why we need hash functions
- How does it work
- **Security properties**
- Algorithms
- Example: The Secure Hash Algorithm SHA-1

Hash Function: Security Properties

- **Preimage Resistance:** For a given output 'z', it is impossible to find any input 'x' such that h(x) = z, i.e., h(x) is **one-way**.
- **Second Preimage Resistance:** Given x_1, and thus $h(x_1)$, it is computationally infeasible to find any x_2 such that $h(x_1) = h(x_2)$.
- **Collision Resistance:** It is computationally infeasible to find any pairs $x_1 \neq x_2$ such that $h(x_1) = h(x_2)$.



The three security properties of hash functions





Message Authentication Codes (MAC's)

Some other MACs!







This is Our MAC!





Content of this Chapter

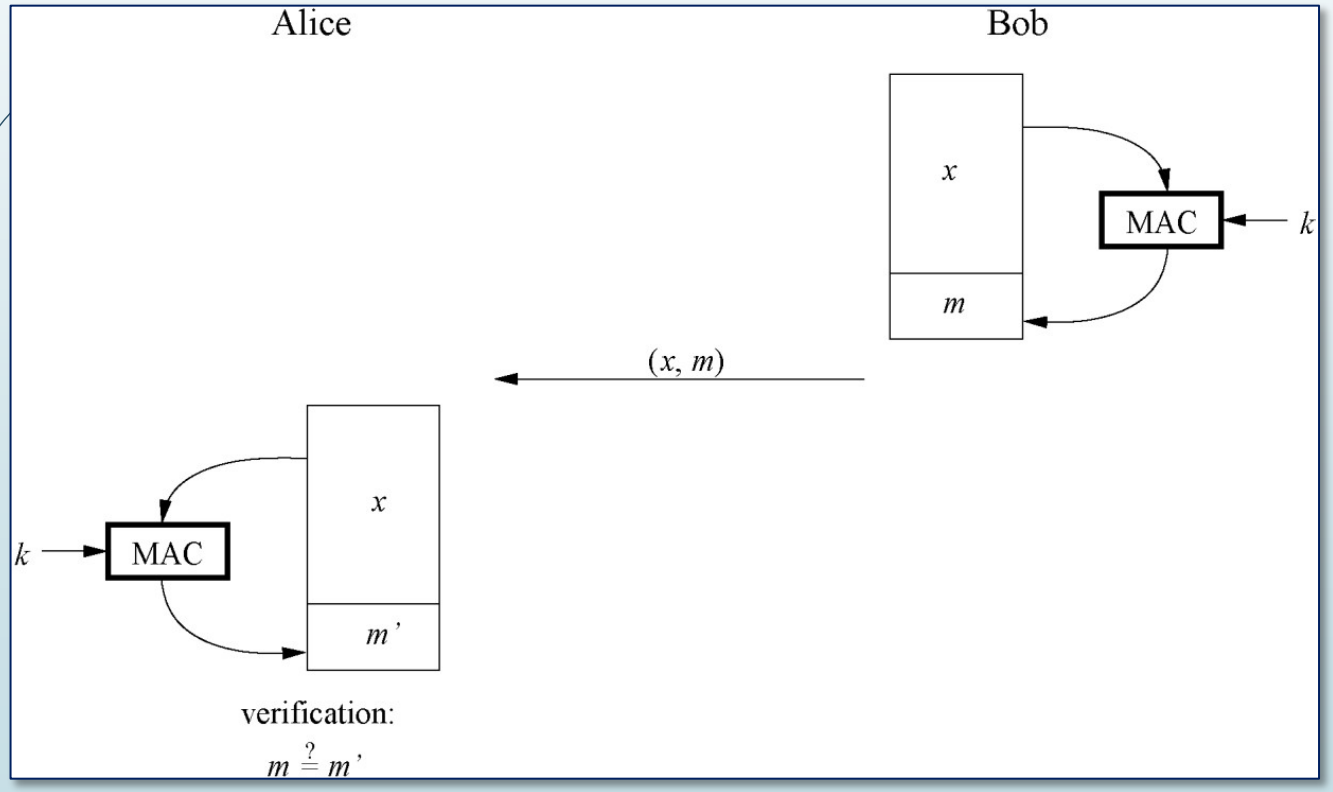
- The principle behind MACs
- The security properties that can be achieved with MACs
- How MACs can be realized with hash functions and with block ciphers

Content of this Chapter

- **The principle behind MACs**
- The security properties that can be achieved with MACs
- How MACs can be realized with hash functions and with block ciphers

Principle of Message Authentication Codes

- Similar to digital signatures, MACs append an authentication tag to a message
- MACs use a **symmetric key** '*k*' for generation and verification
- Computation of a MAC: $m = MAC_k(x)$



Content of this Chapter

- The principle behind MACs
- **The security properties that can be achieved with MACs**
- How MACs can be realized with hash functions and with block ciphers

Properties of Message Authentication Codes

1. **Cryptographic checksum**
   A MAC generates a cryptographically secure authentication tag for a given message.
2. **Symmetric**
   MACs are based on secret symmetric keys. The signing and verifying parties must share a secret key.
3. **Arbitrary message size**
   MACs accept messages of arbitrary length.
4. **Fixed output length**
   MACs generate fixed-size authentication tags.
5. **Message integrity**
   MACs providemessage integrity: Any manipulations of a message during transit will be detected by the receiver.
6. **Message authentication**
   The receiving party is assured of the origin of the message.
7. **No nonrepudiation**
   Since MACs are based on symmetric principles, they do not provide nonrepudiation.